

CS383 - Software Engineering

Software Process Models (I)

Topics to be covered in this lecture

- Software process models
- Generic software process models
- Process activities: specification, development, V&V, and evolution
- Coping with changes

What is a software process model?

- A simplified description of software process activities representation from a particular viewpoint.
- It may contain
 - Sequence of process activities
 - Inputs/outputs to each process activities
 - Roles and responsibility of people involve (clients, developers, administrators)
- It considered as a roadmap to guide software team
- Software processes are categorized into two groups:
 - Plan-driven processes (this lecture)
 - Agile processes (next lecture)

Generic software process models

- Most software process models are based on one of the following paradigms
 - The Waterfall Model
 - Incremental Development
 - Reuse-oriented Development
- Generic process models typically adapted and extended to create specific software process models that can be applied

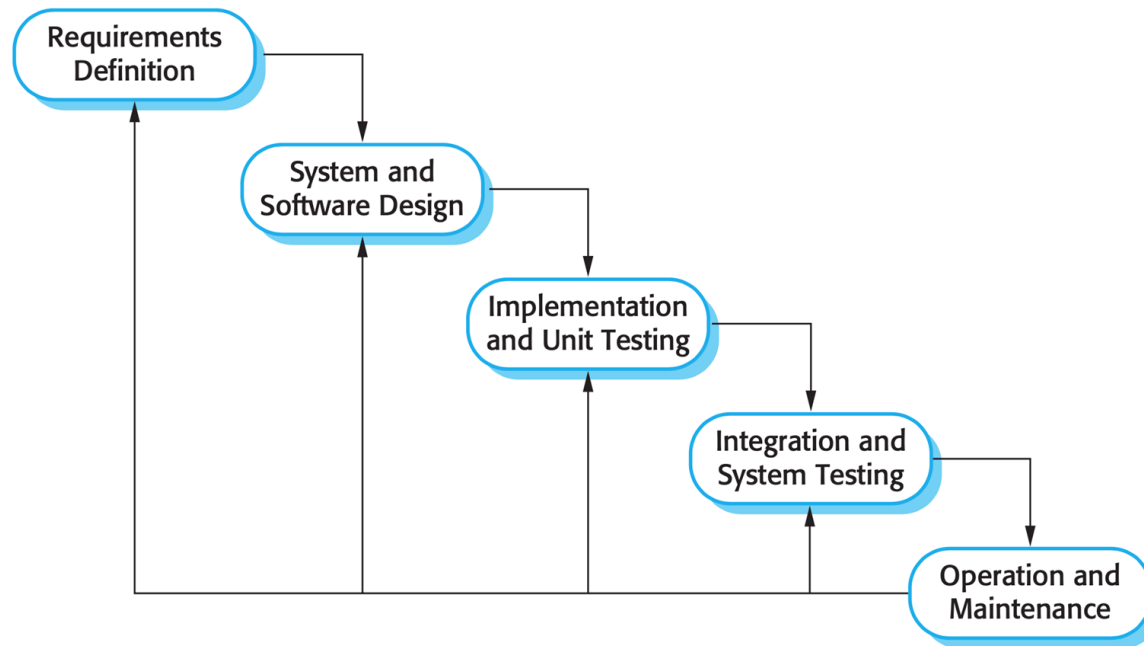
The waterfall model

• Strengths

- Aligns to system engineering process
- Complete set of documentation

• Weaknesses

- Inflexible to changing requirements
- Late discovery of technical problems
- Sequential: next phase waits for previous phase



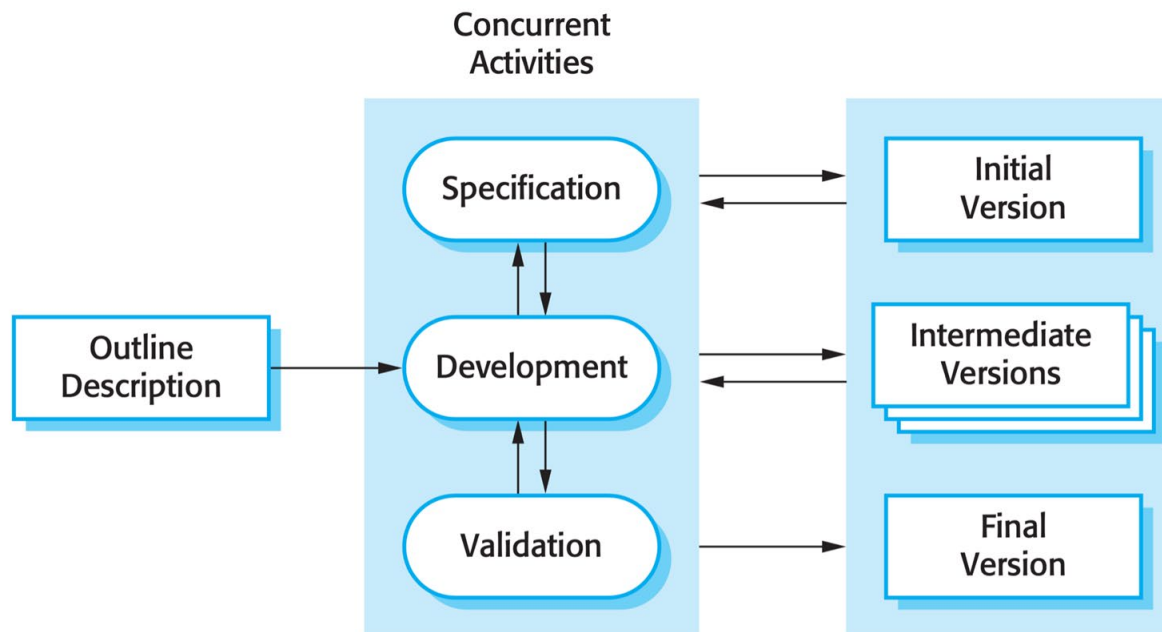
Incremental development

• Strengths

- Effectively manages evolving requirements
- Identifies and resolves technical risk early
- Receive early customer feedback

• Weaknesses

- Reduced visibility and control of activities
- May lead to poor structured software



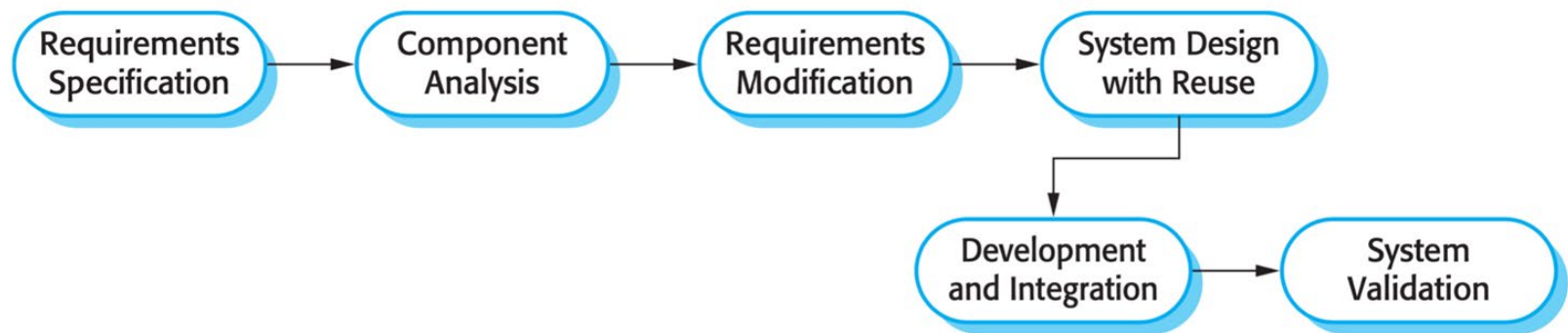
Reuse-oriented (component-based) Development

• Strengths

- Lowers the cost and time of development
- Lowers technical risk

• Weaknesses

- Compromises made on requirements
- Less control over software evolution



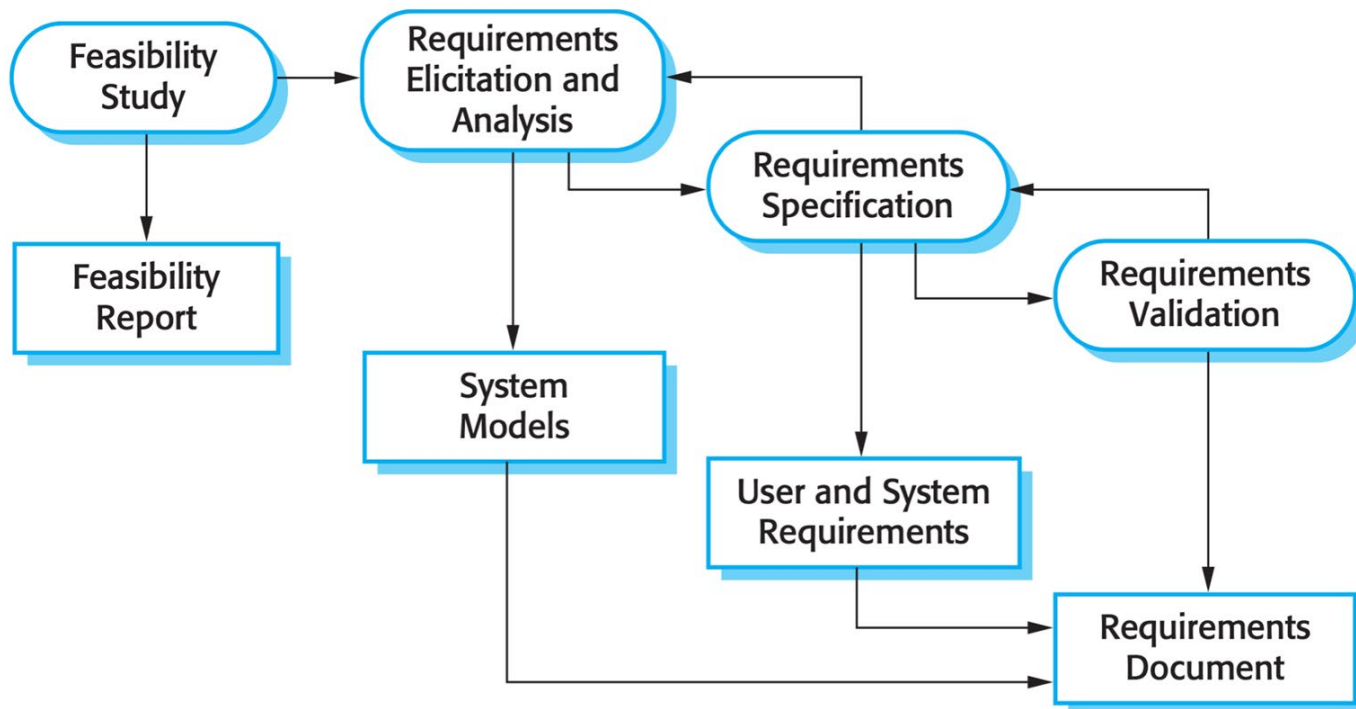
Process activities of software development

In the last, we mentioned fundamental software development activities:

1. Software specification
 2. Software development
 3. Software verification and validation
 4. Software evolution
- Activities are organized differently in development process
 - How to carry out these activities depends on the process model
 - Eg. in waterfall model, they are sequentially organized
 - Eg. in incremental model, they are interleaved and repeated based on development goals

Process activity: software specification

- Also referred to as requirement engineering
- Process of understanding and defining required services
- Also, identifying constraints on services operation and development
- Requirements must be represented in different levels of details (why?).



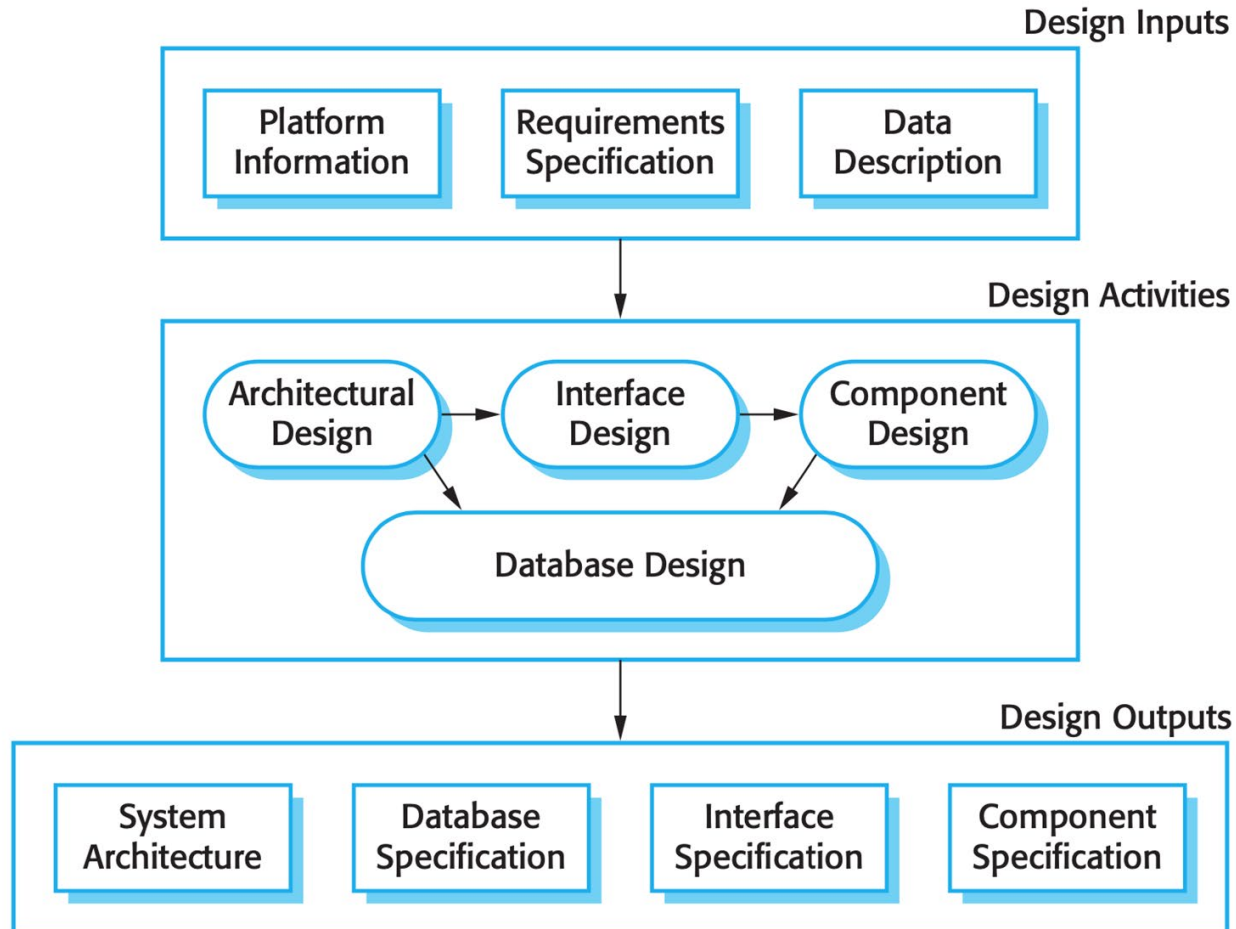
Process activity: software development

As mentioned before, software development is two-fold parts:

1. Software design: a description of the structure of the software to be implemented

- E.g. data models, interfaces of system components, algorithms (sometimes)
- Design is iterative process.
 - Details are added with constant backtracking
- As software (design) there are inputs, design activities, and outputs

Process activity: software design



Process activity: software implementation

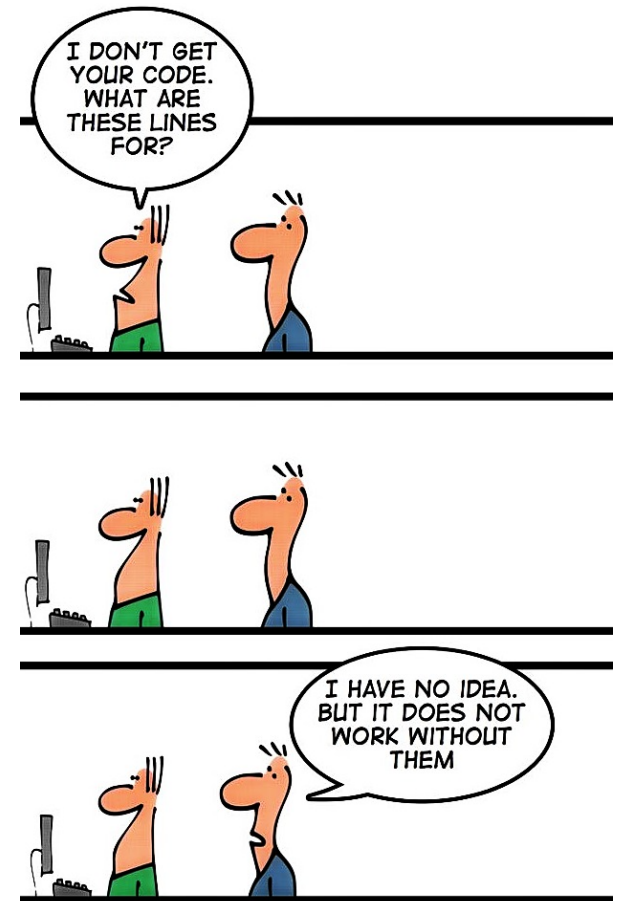
2. Software implementation: processes the design output and converting the specification (outputs of design phase) into executable system

- Manual implementation
- In MDD, model transformation is used to transformed design outputs (graphical models like UML) into other types of software: documents, code, models
- In Agile development: design outputs could be represented in the code of the program



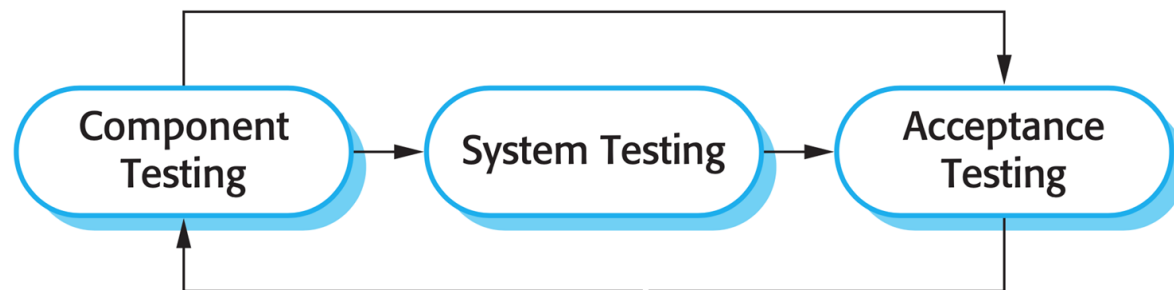
Process activity: software development (testing)

- Software development require preliminary tests
- Defects of development can occur at any level of design or implementation
- Defects cascade: propagate during development or even outside of development
- Testing of development should be iterative process
 - A change of a component can trigger a problem into another



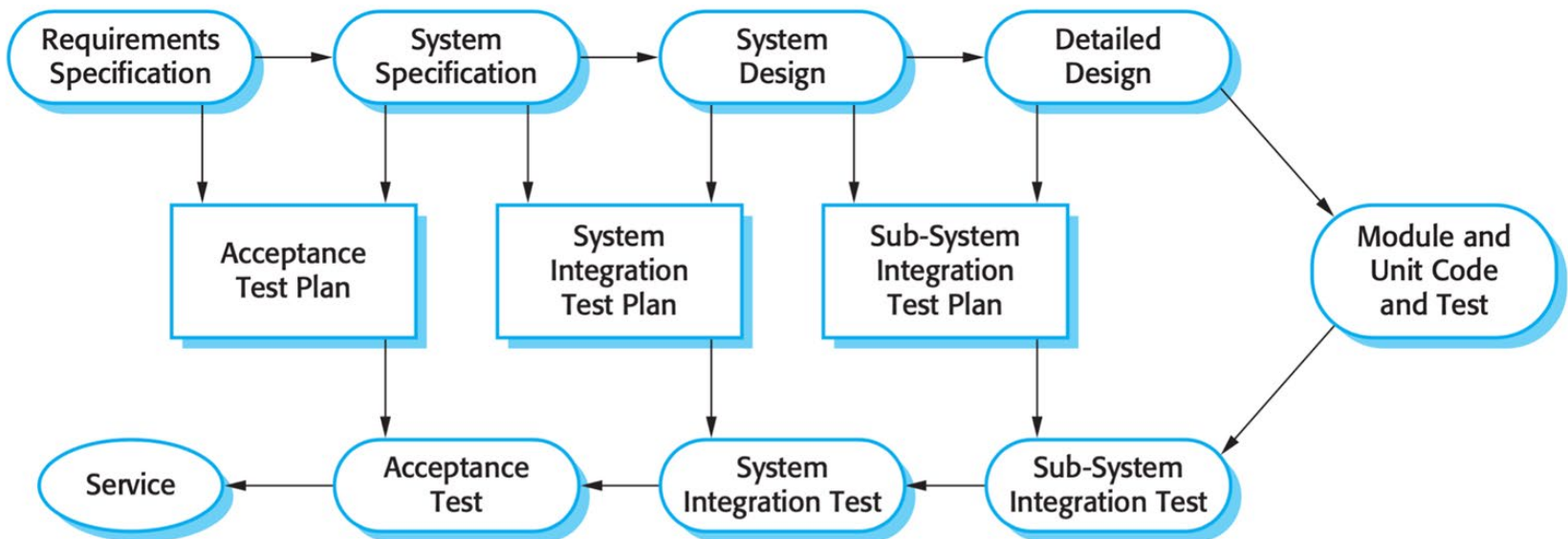
Process activity: software V&V

- Software V&V is a process to ensure that a system conforms of its specifications
- Three-stage testing process should be used
 - Component Testing: test each component independently
 - System testing: system components are integrated and testing as whole
 - This could be multi-staged testing
 - Acceptance Testing: test using real data from customers/clients



Process activity: software V&V (cont.d)

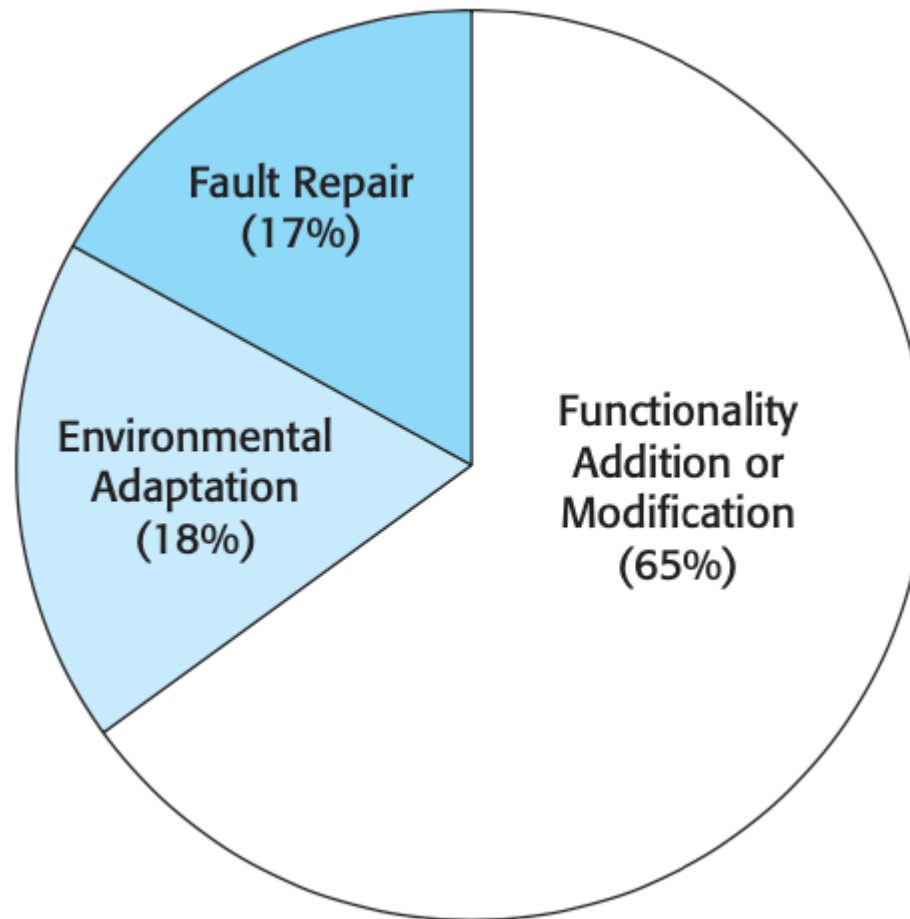
- In plan-driven software process, there are usually test plans: link between testing and development activities
- This model also known as V-model



Process activity: software evolution

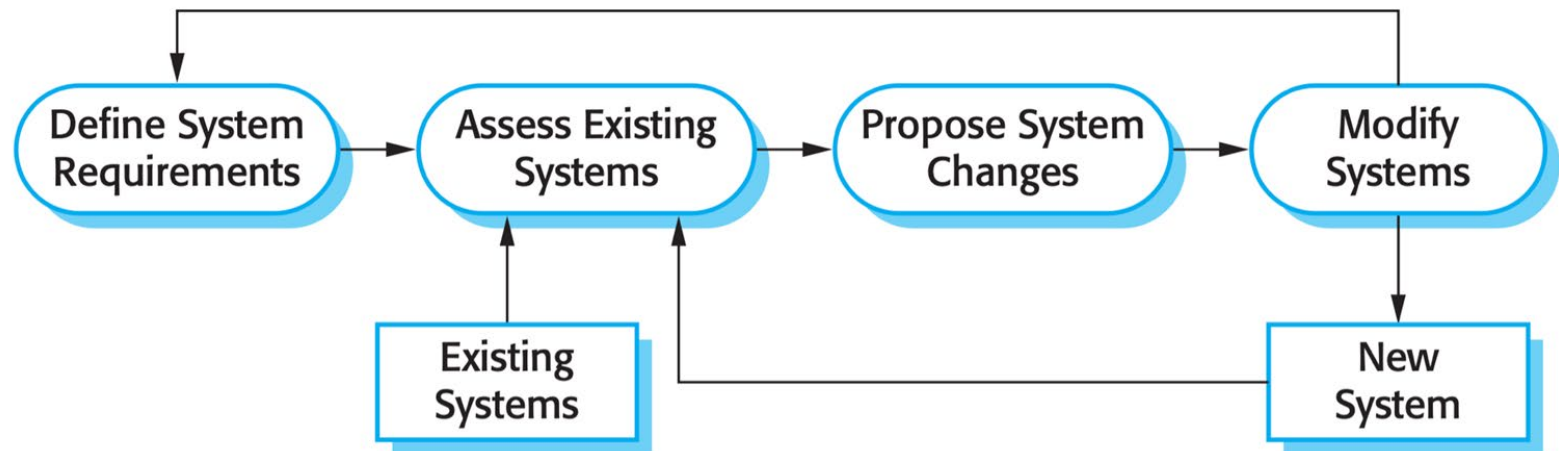
- It is a general process of changing the system after its deployment
- There are three types of software maintenance
 - Fault repairs: correct the software to be valid
 - If coding error: it usually cheap to fix
 - If design error: it is more expensive because it involves several component fixes
 - If requirement error: it is most expensive as it involves redesign of the system
 - Environmental adaptation: aspect of system environment , hardware, support software changes
 - Functional addition: system requirement change to respond to organizational or business change

Process activity: software evolution (cont.d)



Process activity: software evolution (cont.d)

- Typical process of software evolution (maintenance)



Coping with change

- Change is inevitable in any software
 - System requirements change
 - New business needs
 - Technologies evolution
 - New design and implementation emerge
- Change adds development costs because of rework
 - Eg. establish a new requirement analysis because of new requirements emerge
 - Of course, this leads to redesign as well (adds cost) etc.

Coping with change (cont.d)

- Two approaches can be used to overcome costs
 - In other words, anticipating changes and reduce costs
- 1. Change avoidance: software process includes activities that can anticipate possible changes before significant rework is required
- 2. Change tolerance: the process is designed so that changes can be added at relatively low costs

Coping with change (cont.d)

- Two approaches are used to cope with requirements change
 1. System prototyping: support the change avoidance by producing partially required services for early feedback.
 2. Incremental delivery: incrementally deliver to customer targeted features to allow comments and experimentation (and this support change avoidance and change tolerance)